FleetPOD Technical Design Document

Proof of delivery capture, verification, and billing

GitHub URL: Not published yet | Live app: Not deployed

FleetPOD moves proof of delivery closer to the delivery gate so billing can begin sooner. This build covers the core route from driver photo capture through an AI check and human exception handling to a Tally-ready export. The photo remains the source of truth, and a billing executive controls corrections and invoice marking.

1 Product scope and user experience

The interface follows the brief's minimal OLED direction: a true-black canvas, quiet charcoal surfaces, clear gray body text, and distinct green, blue, gold, and purple cues. Drivers see their trips and one large capture action. Billing staff see missing proof, uncertain readings, and verified trips in separate focused views. The layout adapts to mobile widths, and the driver capture screen includes Hindi and Marathi spoken instructions.

Role	Main actions	Access boundary
Driver	Select demo trip, photograph or choose a POD, retry a requested photo, queue capture offline.	Can upload photos and view demo trip details; cannot verify or invoice.
Billing executive	Correct extracted fields, verify or request a retake, export CSV, mark invoices, view dashboard.	Can review and bill trips; every decision is logged.
Client viewer	Open a tokenized, read-only trip link and view its POD image.	Can view one trip only; no edit or list access.

Core flows

- Driver selects a trip, takes a photo, and the browser stamps capture time and optional GPS coordinates. The image is resized and JPEG-compressed below 300 KB.
- If the network is unavailable, the photo and metadata are stored in IndexedDB and sent when the connection returns. The UI shows the queued count.
- The vision check reads the LR number, consignee stamp, quantity, shortage or damage remarks, delivery date, and signature. Missing credentials or a model error sends the POD to human review.
- A clean high-confidence match becomes verified. Shortage or damage, low confidence, missing fields, and mismatches require human review or a retake. Retakes are capped at two.
- A billing executive corrects fields against the original image, then verifies the POD. Verified trips enter the billing queue; staff export a CSV with a POD image link and mark invoices manually.

2 Architecture and data model

Layer	Implementation	Reason
Frontend	Responsive HTML, CSS, and JavaScript served by FastAPI; PWA shell and IndexedDB queue.	A small dependency-free interface loads on basic mobile devices and keeps capture usable during network drops.
Backend	FastAPI JSON routes with signed session cookies and role checks.	Separates driver upload from billing review and keeps validation and decision rules server-side.
Storage	SQLite trip, user, audit, and one-time upload-link tables; private local image directory.	Simple for a solo take-home demo. The persistence boundary is small and can move to managed storage.
Vision AI	Optional OpenAI Responses API call with the compressed photo and a structured field schema.	Produces machine-readable extraction while keeping verification and exception decisions explicit.

Request path: Driver PWA -> FastAPI session and role check -> image validation and compression -> optional vision check -> trip status and audit row -> review queue or billing queue. The offline queue stores pending photo data in the browser and retries after reconnection. Client links use a random per-trip token and return a read-only view.

Entity	Important fields	Relationship / purpose
User	email, display name, role, PBKDF2 salt and digest	Authenticates driver and billing roles.
Trip	client, vehicle, destination, branch, freight, LR, e-way bill, expected quantity, status, timestamps, retake count, image path, AI fields, share token	One trip is the unit of POD capture, review, billing, and client sharing.
Audit log	actor, action, trip ID, detail, UTC timestamp	Records upload, AI result, human decision, CSV export, and invoice marking.
Upload link	random token, trip ID, expiry, used timestamp	Mocks a one-time hired-driver SMS upload link; expires after 24 hours.

3 POD vision check and exception handling

The server sends an uploaded JPEG as a base64 data URL to a vision-capable Responses API model when OPENAI_API_KEY is configured. The request asks for the LR number, stamp visibility, quantity, shortage or damage remarks, delivery date, signature visibility, confidence, and short issues. A JSON schema constrains the response. The request sets store=false.

Condition	System result	Next step
Strong confidence, matching LR and quantity, stamp and signature present, no shortage or damage	Verified	Available for billing export.
Any shortage or damage remark, or a quantity difference	Needs review	Billing executive compares the extracted fields and image.
Low confidence, unreadable or missing required fields, or LR mismatch	Retake requested	Driver takes a clearer photo. After two retakes the case goes to human review.
No API key, API error, invalid model output, or no network	Needs review	No automatic verification; billing executive uses the image as evidence.

The API response never directly approves an invoice. The backend checks the extracted LR and quantity against the trip, applies confidence and exception rules, and stores the original photo. Internal image access requires a signed-in role; a client can retrieve one image through its unguessable read-only trip token. Billing users can edit extracted values and add an audit note.

4 Interface, accessibility, and security

Screen	Design decision
Driver capture	A trip card and a large camera action; optional Hindi or Marathi speech; capture time and GPS shown after selection; no driver text form.
Review queue	Exception reason, original photo, editable fields, confidence, and clear verify or retake actions in one view.
Billing queue	Verified trips only, with LR and e-way bill, freight, POD image, client link, export, and manual invoice marking.
Missing-POD dashboard	Four key metrics, follow-up table, and branch-level outstanding counts; tables scroll horizontally on small phones.

- Body copy uses light gray on black or charcoal. Accent colors are reserved for status and action cues; focus outlines remain white and visible.
- Reduced-motion preferences disable nonessential animation. Icon-only controls have accessible names. Input validation is performed in the browser and server.
- Passwords use PBKDF2-HMAC-SHA256 with unique random salts. Sessions use signed, same-site cookies. Role-protected routes gate review, export, and invoice actions. The API key comes from the environment, `.env` is ignored by Git, and demo data is synthetic.
- Uploads accept supported image types, are decoded and re-encoded server-side, and are capped at 300 KB. Security headers limit framing, MIME sniffing, and content sources.

5 Tests and operational limits

The test suite covers authentication, role boundaries, the offline app shell, no-key fallback, exception classification, the two-retake limit, human review to billing export, one-time upload links, and invoice timing. The recorded run completed with 8 passing tests.

Assumptions

- OpenAI was selected as the vision provider because the brief does not name a provider.
- Additional LR and e-way bill values, branch assignments, trip dates, quantities, demo account names, and POD images are synthetic values created for this prototype.
- A verified POD may enter the billing queue only after a clean machine check or a billing executive's explicit verification.
- The client share link is a single-trip read-only token link. API failure or missing credentials must prefer review over automatic approval.

Cut for this build

Live SMS and Tally integrations, daily trip CSV import, real client contact details, production hosting, live GPS tracking, trip planning, and fuel or driver-expense accounting are out of this build. SMS is represented by a single-use upload link; Tally is represented by a downloadable CSV.

Next steps

Publish the repository and hosted demo, replace SQLite and local files with managed production services, add daily trip import, agree retention and deletion rules for POD images, validate branch access and audit retention, and test the vision rules against an approved set of real-format but privacy-cleared documents.

References

OpenAI Images and Vision: https://developers.openai.com/api/docs/guides/images-vision | Structured Outputs: https://developers.openai.com/api/docs/guides/structured-outputs